

Unit 06: Data link layer – error control and flow control

CONTENTS

Objectives

Introduction

- 6.1 What is the block coding?
- 6.2 Error Control Mechanisms May Work in Two Different Ways
- 6.3 Error Correction
- 6.4 Data Link Control
- 6.5 Flow Control and Error Control
- 6.6 Protocols
- 6.7 NOISELESS CHANNELS
- 6.8 NOISY CHANNELS

Self-Assessment

Answers: Self-Assessment

Summary

Keywords

Review Questions

Objectives

- what is block coding
- learn different types of error detection mechanism
- Different types of framing techniques

Introduction

When transmitting data from source to destination, it is likely that the data will get compromised during transmission. Your bits shift from one to zero or zero to one between transmissions as seen in Figure 1.

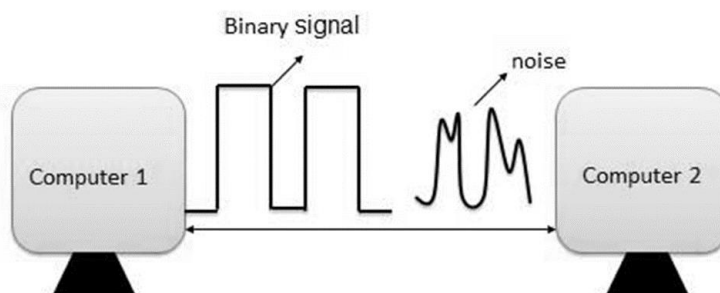


Figure 1 Error

So, how do we prove that my pieces aren't tampered with? or How would the receiver be able to tell whether the bits produce an error? As a result, we must learn the pathways for detecting errors. As a result, data can be manipulated when being transmitted. This is the reason we need to learn error detection mechanism.

6.1 What is the block coding?

If I want my receiver to know whether the bits are corrupted or not, I use block coding, which divides our messages into a block called a data term. So, along with the data terms, we'll add some redundant bits. When we combine the redundant terms with the data universe, we get our code name. If I write 101, I'm using data terms. This is a list of my data terms. Now, if we add r redundant bits to this data word, it becomes the code word; if we add r redundant bits to this data word, it becomes the code word.

This is referred to as a code term. As a result, the sender sends these bits to the recipient. As a result, we'll use these redundant bits to determine if my data is properly corrected and the same data that the sender sent, or if my bits are incorrect. So we can build a combination of two raise to power k data words using k bit. With n bits, we can make a dataword that combines two raise to power n datawords.

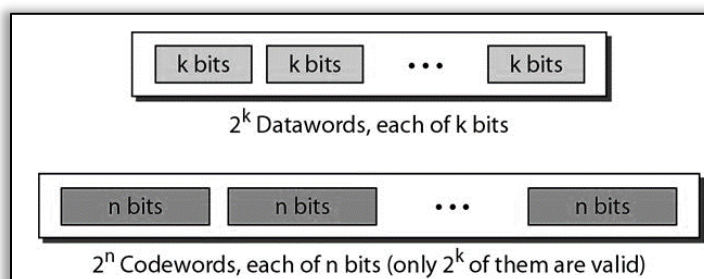


Figure 2 Block Coding

Ques:- How can errors be detected by using block coding?

Ans:- So, we have two options for determining if my data is right or not: my potential recipient already has a valid codeword, and that valid code will determine if my data is correct or not. If they don't align, the bits are corrupted, and the original code word has been replaced by an invalid one as seen in Figure 3.

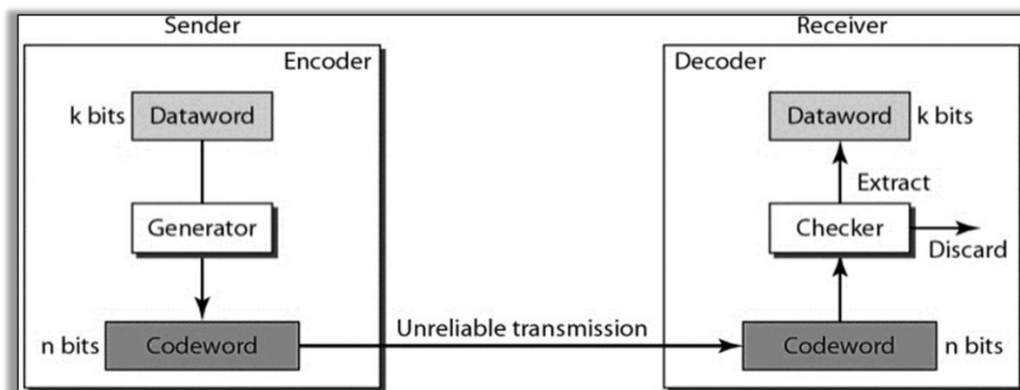


Figure 3 Error Detection

So, let's start with a few topics that are directly or indirectly relevant to error detection and correction mechanisms. First and foremost, we use only a small number of integers while we are using modular arithmetic. As a result, we define a modulus, which is a spectrum of zero to n minus one. If n is 12, the range would be zero to 12 minus one, which is zero to 11. As a result, we only use the integers 0 to 11 inclusive. Only integers in the range of zero to n minus one are used in modulo- N arithmetic. In modulus arithmetic, there is no carry and if you do a subtraction there is no carry even if you do an addition there is no carry, I hope you people have seen this, in case of addition and subtraction the outcome is almost the same, the result is almost the same zero plus 0 and zero both are the same here, in case of addition and subtraction the result is almost the same, the result is almost the same zero plus 0. Whenever we use the error correction schemes, we'll use the exclusive OR estate. So you must understand what an exclusive is. If the bits are the same, the result will be zero; if the bits are different, the result will be one. Similarly, these are the results of the exclusive OR, and this is what

we'll use to assist you in error detecting mechanisms, such as when calculating the cyclic redundancy search. Even so, in addition to performing, we use this XOR property.

Types of Error

There are bits that can be changed during transmissions whenever a bit flow from one place to another. So, single bit error, multi bit error, and burst error are the three categories of errors as seen in Figure 4.

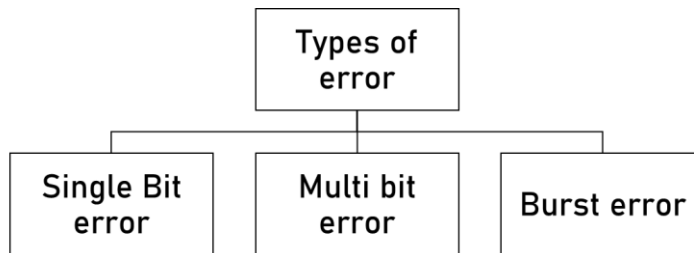


Figure 4 Types of Error

Single Bit Error

If only a single bit is corrupted, you can see that only a single bit from zero to one is corrupted. As a result, this is a well-known example of a single bit as seen in Figure 5.

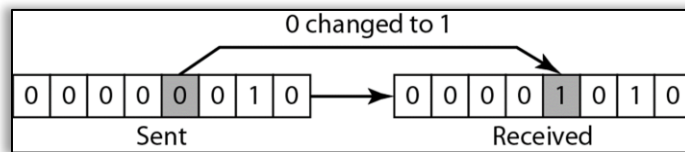


Figure 5 Single bit Error

Multi-bit Error

If two bits in two separate places are corrupted. As a result, this is referred to as a multi bit malfunction.

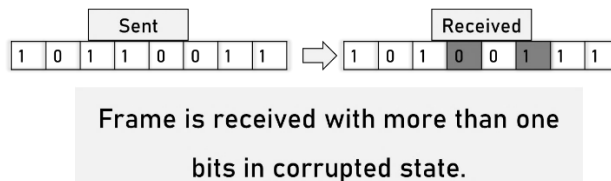


Figure 6 Multi-bit Error

Burst Error

A third type of error is a burst error, which occurs when several bits are scrambled at the same time. The fact that we have 100 instead of 011 indicates that three consecutive bits are compromised. As a result, this is referred to as a burst malfunction.

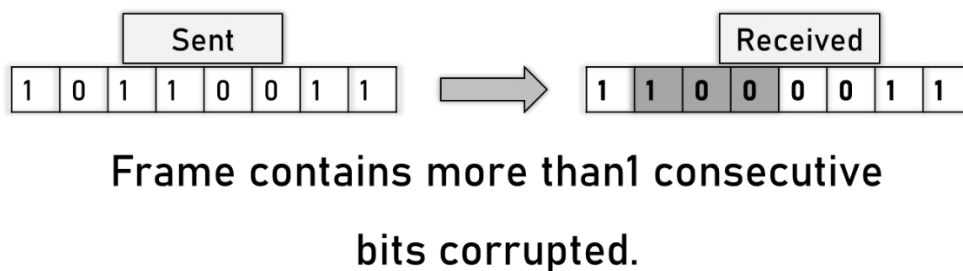


Figure 7 Burst Error

So, single bit error, multi bit error, and burst error are the three categories of errors. For several bits, just a single bit is compromised. When more than one bit is corrupted in multiple places, this is referred to as a multi bit error. When consecutive bits are corrupted, this is referred to as a burst error.

Redundancy

We add the redundant bits together with the data if we have a data expression. As a result, these redundant bits assist the receiver in determining if the bits are corrupted or false. So these are the pieces that assist the receiver in determining the outcome. We have a list of data words. This is referred to as data terms, and it isn't real data. As a result, it can be made up of 8 bits or any other number of bits. We then combine the redundant bits with the data. As a result, this amount becomes my code term. So we're sending duplicate bits, which means we're sending extra bits.

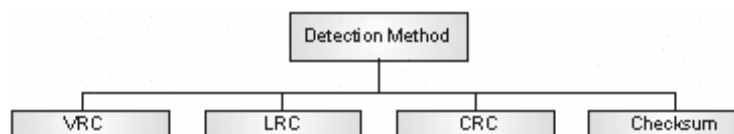


Figure 8 Four Types of Redundancy Check

Vertical Redundancy Check: The vertical redundancy check (VCR), also known as a parity check, is the most effective and least costly method for error detection. Any data unit is appended with a redundant bit called a parity bit in this procedure, bringing the total number of bits in the unit (including the parity bit) to an even number.

Longitudinal Redundancy Check: In longitudinal redundancy check (LRC), a block of bits is organized in a table (rows and columns). For example, instead of sending a block of 32 bits, we organize them in a table made of four rows and eight columns, as shown in Figure 9. We then calculate the parity bit for each column and create a new row of eight bits, which are the parity bits for the whole block.

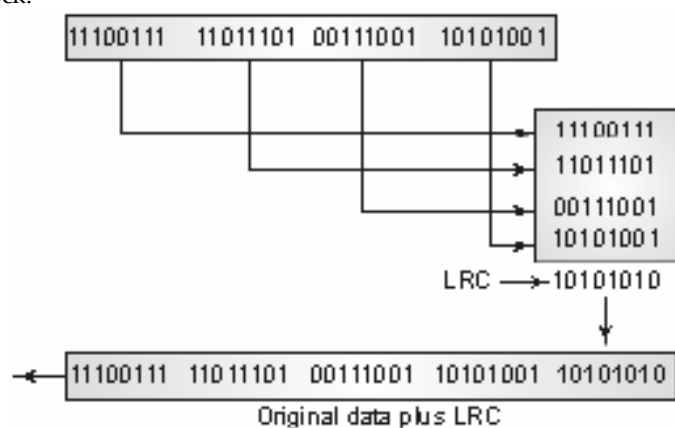


Figure 9 LRC

CRC and checksum will discuss later in this chapter.

6.2 Error Control Mechanisms May Work in Two Different Ways

We have two options for detecting and correcting a mistake. So we have two mechanisms: one is called error detecting, and the other is called error correction as seen in Figure 10. To begin, there is a distinction to be made between error detection and error correction. We only consider whether the error has occurred or not in the case of error detection; we do not consider which bit has an error, the magnitude of the error, whether it is a single bit error or a burst error in the case of error detection, but we do in the case of error correction.

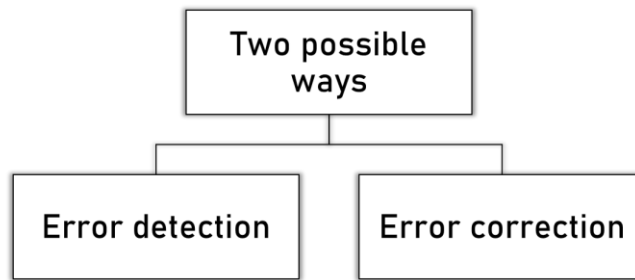


Figure 10 Error Control Mechanism

In the case of error correction, the receiver needs to see how many bits and how big the error is. As a result, in order to correct an error, we must know the precise number of bits that contain the error. We also need to know where the error is located, because the number of errors and their magnitude are critical factors in error correction.

Detection vs Correction

Table 1 Detection vs Correction

Error Detection	Error Correction
we are looking only to see if any error has occurred.	we need to know the exact number of bits that are corrupted and more importantly, their location in the message
Not even interested in the number of errors	The number of the errors and the size of the message are important factors
A single-bit error is the same for us as a burst error	If we need to correct one single error in an 8-bit data unit, we need to consider eight possible error locations;

Error Detection

Errors in the received frames are detected by means of

- Simple Parity Check
- Two-Dimensional parity Check
- Checksum
- and Cyclic Redundancy Check (CRC).

Simply Parity Check

Let's go over each one by one. One is a quick parity check; it may be even or odd parity; in this case, we'll count the number of ones. If I'm counting, because I want equal parity, I'll calculate the amount of one and add to make it even. Let me demonstrate with an example how we can add the redundant bit as seen in Figure 11.

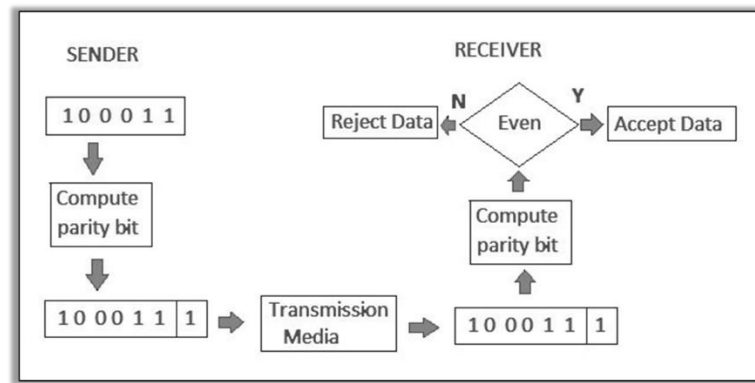


Figure 11 Simple Parity Check

Single Parity Bit

Single parity screening is an easy and inexpensive mechanism for detecting errors. A redundant bit, also known as a parity bit, is appended to the end of the data unit in this technique to make the number of 1s equal. As a result, the cumulative number of bits sent will be nine. If the number of 1s bits is odd, parity bit 1 is appended at the end of the data unit; if the number of 1s bits is even, parity bit 0 is appended. The parity bit is determined from the received data bits and added to the received parity bit at the receiving end. Even-parity checking is a procedure that produces a cumulative number of 1s that is even.

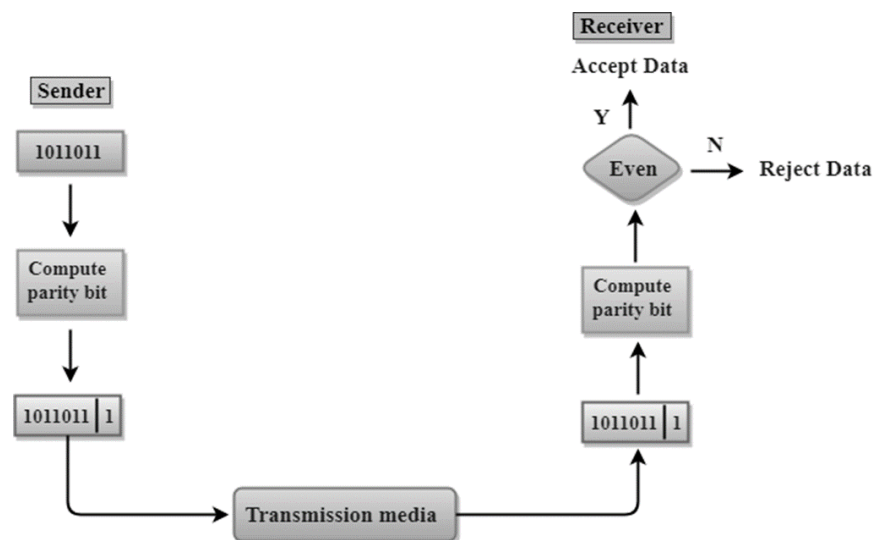


Figure 12 Single Parity Bit

Drawbacks of Single Parity Checking

It can only detect single-bit errors which are very rare.

If two bits are interchanged, then it cannot detect the errors.

Two-Dimensional Parity Check

Two-Dimensional Parity Check, which organizes data in the form of a table, will increase performance.

For each row, parity check bits are computed, which is analogous to a single-parity check.

A block of bits is separated into rows in a Two-Dimensional Parity scan, and the redundant row of bits is applied to the whole block.

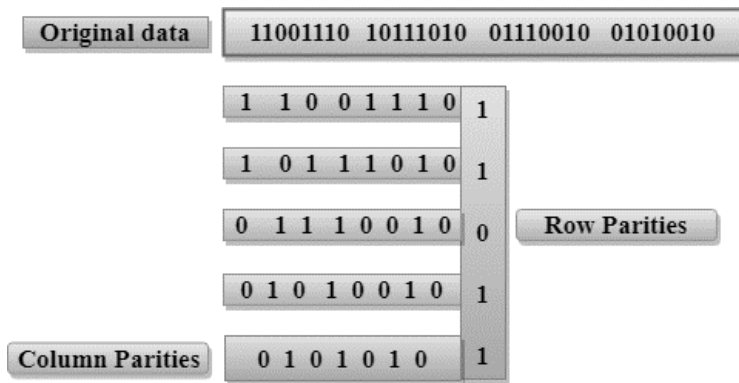


Figure 13 Two-Dimensional Parity Check

Drawbacks Of 2D Parity Check

The 2D Parity checker would not be able to spot the error if two bits in one data unit are corrupted and two bits in the same place in another data unit are not corrupted.

In certain instances, this technique may not be able to spot 4-bit or more errors.

Checksum

A checksum is a method for detecting errors that is based on the principle of redundancy.

It is divided into two parts:

Checksum Generator

A Checksum is generated at the sending side. Checksum generator subdivides the data into equal segments of n bits each, and all these segments are added together by using one's complement arithmetic. The sum is complemented and appended to the original data, known as checksum field. The extended data is transmitted across the network.

Suppose L is the total sum of the data segments, then the checksum would be $?L$

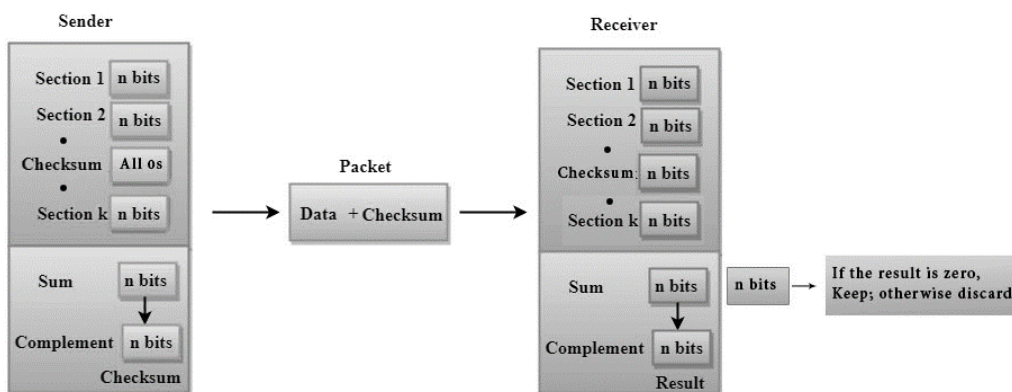


Figure 14 Checksum

The Sender follows the given steps as shown in Figure 14:

The block unit is divided into k sections, and each of n bits.

All the k sections are added together by using one's complement to get the sum.

The sum is complemented and it becomes the checksum field.

The original data and checksum field are sent across the network.

Checksum Checker

A Checksum is verified at the receiving side. The receiver subdivides the incoming data into equal segments of n bits each, and all these segments are added together, and then this sum is

complemented. If the complement of the sum is zero, then the data is accepted otherwise data is rejected.

The Receiver follows the given steps:

The block unit is divided into k sections and each of n bits.

All the k sections are added together by using one's complement algorithm to get the sum.

The sum is complemented.

If the result of the sum is zero, then the data is accepted otherwise the data is discarded.

Cyclic Redundancy Check (CRC)

CRC is a redundancy error technique used to determine the error.

Following are the steps used in CRC for error detection:

In CRC technique, a string of n 0s is appended to the data unit, and this n number is less than the number of bits in a predetermined number, known as divisor which is $n+1$ bits.

Secondly, the newly extended data is divided by a divisor using a process is known as binary division. The remainder generated from this division is known as CRC remainder.

Thirdly, the CRC remainder replaces the appended 0s at the end of the original data. This newly generated unit is sent to the receiver.

The receiver receives the data followed by the CRC remainder. The receiver will treat this whole unit as a single unit, and it is divided by the same divisor that was used to find the CRC remainder.

If the resultant of this division is zero which means that it has no error, and the data is accepted.

If the resultant of this division is not zero which means that the data consists of an error. Therefore, the data is discarded.

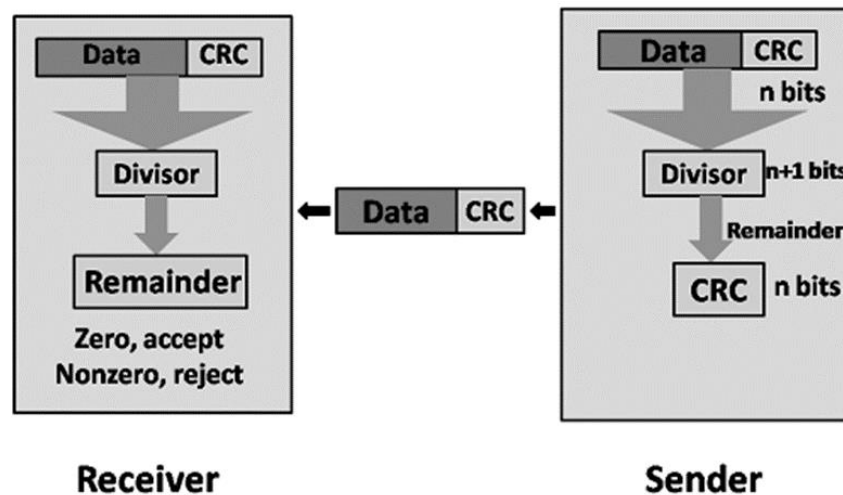


Figure 15 Cyclic Redundancy Check

Let's look at an example to better explain this concept:

Assume the divisor is 1001 and the initial data is 11100.

CRC Generator

- A CRC generator uses a modulo-2 division. Firstly, three zeroes are appended at the end of the data as the length of the divisor is 4 and we know that the length of the string 0s to be appended is always one less than the length of the divisor.
- Now, the string becomes 11100000, and the resultant string is divided by the divisor 1001.
- The remainder generated from the binary division is known as CRC remainder. The generated value of the CRC remainder is 111.

- CRC remainder replaces the appended string of 0s at the end of the data unit, and the final string would be 11100111 which is sent across the network as seen in Figure 16.

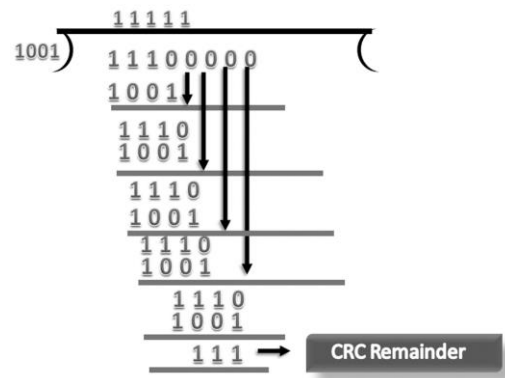


Figure 16 CRC Generator

CRC Checker

- The functionality of the CRC checker is similar to the CRC generator.
- When the string 11100111 is received at the receiving end, then CRC checker performs the modulo-2 division.
- A string is divided by the same divisor, i.e., 1001.
- In this case, CRC checker generates the remainder of zero. Therefore, the data is accepted as seen in Figure 17.

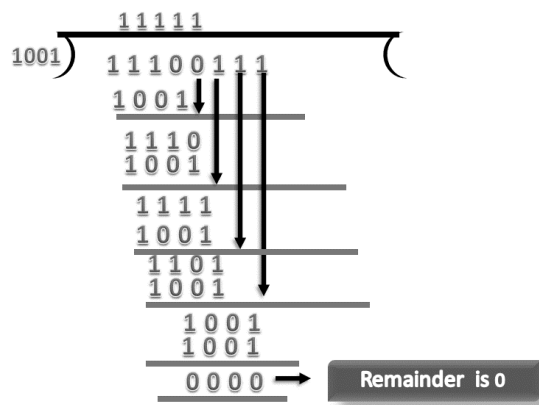


Figure 17 CRC Checker

6.3 Error Correction

Error correction is much more complex than error prediction, as we previously said. The receiver just needs to realize that the sent codeword is invalid in error detection; in error correction, the receiver must locate (or guess) the original codeword sent. We may assume that error correction requires more redundant bits than error detection.

Errors and Errors correcting code

In error correction, the encoder and decoder have different structures.

Bits can be corrupted when being transferred over a data network due to congestion and network issues. Errors are caused by distorted bits, which cause the recipient to receive erroneous data. Under the limits of the algorithm, error correcting codes determine the precise number of bits that have been corrupted and the origin of the corrupted bits. Error-correcting codes (ECC) are a series of numbers

developed by algorithms for detecting and correcting errors in data transmitted over a noisy medium.

Error-correcting codes can be divided into two categories.

Block codes-

The message is broken down into fixed-size chunks of bits, with redundant bits inserted for error detection and correction.

Convolutional codes use data streams of arbitrary length to create parity symbols, which are created by sliding a Boolean function over the data stream.

Hamming Distance

So, the first is our hamming distance, which is the number of places where the related symbols vary from two strings of equal length. But, using hamming distance, we'll figure out how many bits are distinct.

Example

Find the Hamming Distance Between Two Pairs of Words

Ques:- The Hamming distance $d(000, 011)$ is

Ans:- $\begin{array}{c} 000 \\ 011 \end{array} = 2$ (two 1s)

$\begin{array}{c} 10101 \\ 11110 \end{array} = 3$ (three 1s)

Minimum Hamming Distance

The shortest Hamming distance for all possible pairs of strings in a series of equal length strings is called the minimum Hamming distance.

Example

Suppose there are four strings 010, 011, 101 and 111.

$010 \oplus 011 = 001$, $d(010, 011) = 1$.

$010 \oplus 101 = 111$, $d(010, 101) = 3$.

$010 \oplus 111 = 101$, $d(010, 111) = 2$.

$011 \oplus 101 = 110$, $d(011, 101) = 2$.

$011 \oplus 111 = 100$, $d(011, 111) = 1$.

$101 \oplus 111 = 010$, $d(101, 111) = 1$.

Hence, the Minimum Hamming Distance, $d_{min} = 1$.

Hamming Code

Hamming code is a block code that can identify and correct single-bit errors while detecting up to two simultaneous bit errors. It was created by R.W. Hamming for the purpose of error correction. The source encodes the message using this coding process by adding redundant bits into the message. Extra bits are produced and placed at unique locations in the message to allow error detection and correction. When the destination receives this message, it performs recalculations in order to locate errors and determine which bit location is incorrect.

Hamming Code is a method of encoding a letter.

The procedure used by the sender to encode the message encompasses the following steps –

Step 1 – Calculation of the number of redundant bits.

Step 2 – Positioning the redundant bits.

Step 3 – Calculating the values of each redundant bit.

Once the redundant bits are embedded within the message, this is sent to the user.

Decoding a Hamming Code message

When a message is sent, the recipient performs recalculations to find and correct errors. The recalculation procedure is as follows:

Step 1: Determine how many redundant bits there are.

Step 2: Putting the obsolete pieces in their proper places.

Step 3: Testing for parity.

Step 4: Identifying and correcting errors

Step 1 – Calculation of the number of redundant bits

Using the same formula as in encoding, the number of redundant bits are ascertained.

$2^r \geq m + r + 1$ where m is the number of data bits and r is the number of redundant bits.

Step 2 – Positioning the redundant bits

The r redundant bits placed at bit positions of powers of 2, i.e. 1, 2, 4, 8, 16 etc.

Step 3 – Parity checking

Parity bits are calculated based upon the data bits and the redundant bits using the same rule as during generation of c_1, c_2, c_3, c_4 etc. Thus

$c_1 = \text{parity}(1, 3, 5, 7, 9, 11 \text{ and so on})$

$c_2 = \text{parity}(2, 3, 6, 7, 10, 11 \text{ and so on})$

$c_3 = \text{parity}(4-7, 12-15, 20-23 \text{ and so on})$

Step 4 – Error detection and correction

The decimal equivalent of the parity bits binary values is calculated. If it is 0, there is no error. Otherwise, the decimal value gives the bit position which has error. For example, if $c_1c_2c_3c_4 = 1001$, it implies that the data bit at position 9, decimal equivalent of 1001, has error. The bit is flipped to get the correct message.

6.4 Data Link Control

Data link management and media access control are the two primary features of the data link layer as shown in Figure 18. The first, data link control, is concerned with the architecture and procedures for node-to-node connectivity between two neighbouring nodes. This feature is covered in this chapter. The data link layer's second feature is media access control, or how to share the link.

Framing, flow and error management, and software-implemented protocols are all data link control functions that ensure seamless and stable frame transfer between nodes. The first topic in this chapter is framing, or how to arrange the pieces held by the physical layer. The flow and error management are then discussed.

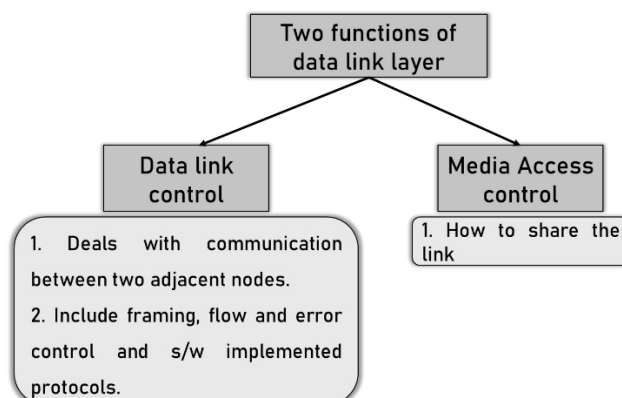


Figure 18 Data Link control

FRAMING

Moving bits in the form of a signal from the source to the destination is what data transfer in the physical layer entails. Bit synchronisation is provided by the physical layer, which ensures that the sender and recipient use the same bit durations and timing.

The data link layer, on the other hand, must cram bits into frames in such a way that each frame can be distinguished from the previous one. Framing is a technique used by our mail department. The basic act of sealing a letter in an envelope divides one piece of data from another; the envelope acts as a delimiter. By inserting a sender address and a recipient address, framing in the data link layer divides a message from one source to a destination, or from other messages to other destinations. The sender address aids the receiver in acknowledging the arrival of the packet; the destination address determines where the packet will go.

While the whole message could fit in one frame, this is rarely achieved. One explanation for this is that frames can be very broad, making flow and error control inefficient. When a message is sent in a single big frame, then a single bit error requires the whole message to be resent. A single-bit error impacts only the small frame when a message is split into bigger frames.

Two Types of Framing

- A. Fixed sized framing
- B. Variable sized framing

Fixed-Size Framing

The size of the frames may be constant or variable. There is no need to define the limits of the frames in fixed-size framing; the size will serve as a delimiter. The ATM wide-area network, which uses fixed-size frames called cells, is an example of this method of framing.

Variable-Size Framing

In variable-size framing, which is common in local area networks. We need a way to identify the end of one frame and the start of the next in variable-size framing. A character-oriented approach and a bit-oriented approach have also been used in the past for this reason.

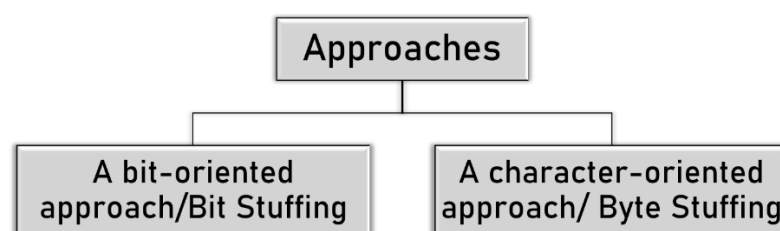


Figure 19 Variable Size Framing

Character Oriented Protocols

The data to be transported in a character-oriented protocol is 8-bit characters from an encoding scheme such as ASCII (see Appendix A). The header, which typically contains the source and destination addresses as well as other control information, and the trailer, which contains redundant bits for error detection and correction, are both multiples of 8 bits. An 8-bit (1-byte) flag is applied to the beginning and end of each frame to distinguish it from the next. The start or end of a frame is signaled by the flag, which is made up of protocol-dependent special characters. The format of a frame in a character-oriented protocol is seen in Figure 20.

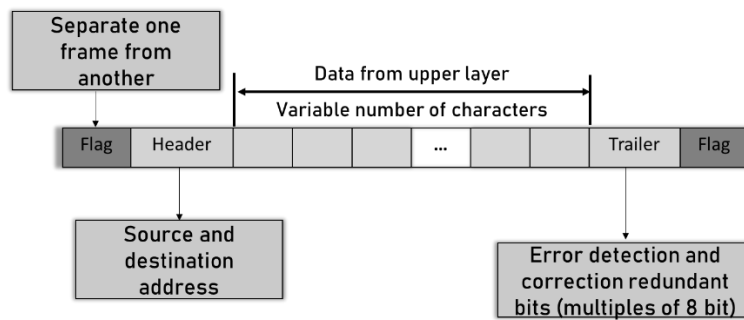


Figure 20 A frame in a character-oriented protocols

When the data link layers only exchanged text, character-oriented framing was common. Every character that isn't used in text that may be chosen as the flag. Other types of content, such as graphs, audio, and video, are now sent. Any pattern used on the flag may also be included in the data. If this occurs, the receiver may believe it has reached the end of the frame when it finds this pattern in the centre of the data. A byte-stuffing technique was applied to character-oriented framing to solve this issue. When a character has the same pattern as the flag, byte stuffing (or character stuffing) adds a separate byte to the data portion of the frame. An extra byte has been stuffed into the data row. This byte, which has a predefined bit pattern, is known as the escape character (ESC). The receiver extracts the ESC character from the data segment and considers the next character as data rather than a delimiting flag whenever it sees it.

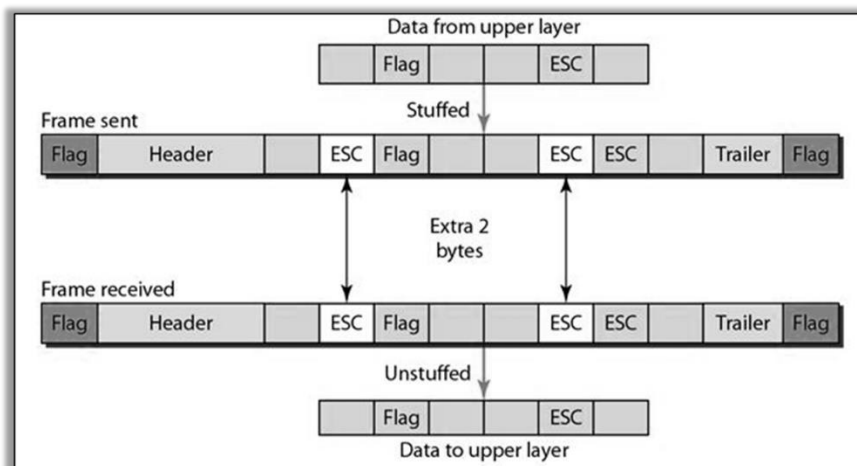


Figure 21 Byte stuffing and unstuffing

The inclusion of the flag in the data section of the frame is allowed by byte stuffing by the escape character, but it introduces a new challenge. What happens if one or more escape characters are accompanied by a flag in the text? The receiver discards the escape character but holds the flag, which is mistakenly read as the frame's end. The escape characters that are part of the text must also be labelled by another escape character to solve this dilemma. To put it another way, if the escape character is already present in the code, an additional one is inserted to indicate that the second one is still present. The scenario is depicted in Figure 21. Another issue with data communications is character-oriented protocols. Unicode and other universal coding schemes use 16-bit and 32-bit characters, which clash with 8-bit characters.

Bit-Oriented Protocols

The data segment of a frame in a bit-oriented protocol is a series of bits that the upper layer interprets as text, image, audio, video, and so on. We do need a delimiter to distinguish one frame from the next, in addition to headers (and perhaps trailers). To specify the beginning and end of the frame, most protocols use the special 8-bit pattern flag 01111110 as the delimiter as shown in Figure 22.

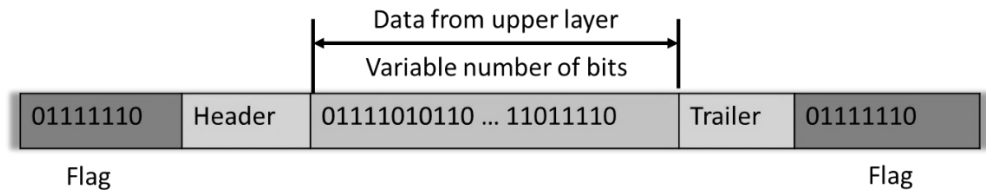


Figure 22 Bit-Oriented Protocol

This flag has the same potential for causing problems as the byte-oriented protocols. That is, if the flag pattern exists in the details, we must notify the receiver that the frame is not yet complete. To avoid the pattern appearing like a flag, we stuff one single bit (rather than one byte).

The receiver finally removes the extra stuffed bit from the results. It's worth noting that the extra bit is inserted after one 0 and five 1s, regardless of the next bit's value. This ensures that the flag field sequence does not appear in the frame by accident.

Bit stuffing at the sender and bit elimination at the receiver are shown in Figure 23. It's worth noting that even though we end up with a 0 after five 1s, we still stuff an 0. The receiver will delete the zero.

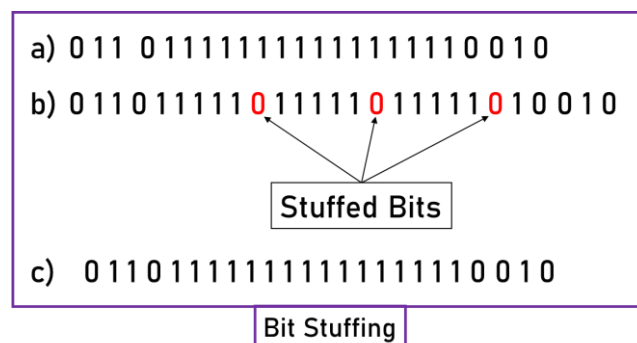


Figure 23 Bit Stuffing and Unstuffing

6.5 Flow Control and Error Control

Data transmission necessitates the cooperation of at least two computers, one of which sends data and the other of which receives it. Even such a simple arrangement necessitates a great deal of planning in order for an understandable transaction to take place. Flow control and error control are the two most critical functions of the data link network. These functions are collectively referred to as data link management. The basic functions of data link layer are framing, Error control and flow control as seen in Figure 24.

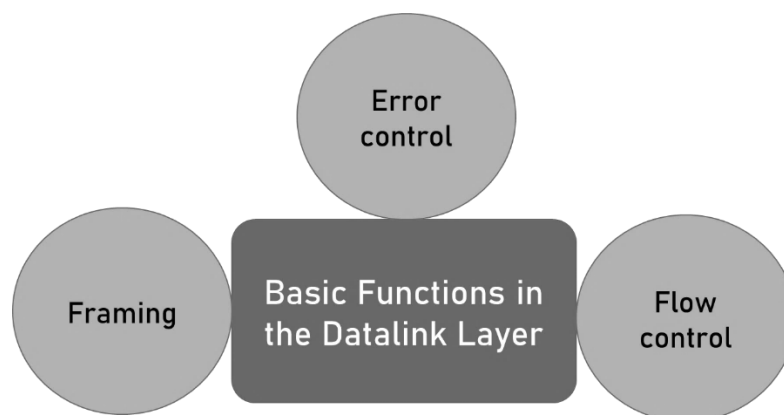


Figure 24 Basic Functions of Data Link Layer

6.6 Protocols

Let's take a look at how the data link layer can use framing, flow management, and error control to deliver data from one node to another. In most cases, the protocols are implemented in software using one of the standard programming languages. To keep our discussions language-free, we wrote a version of each protocol in pseudocode that focuses mostly on the process rather than delving into the nuances of language rules.

Types of Data Link Protocols

The protocols are divided into those that can be used for noiseless (error-free) channels and those that can be used for noisy (error-producing) channels. The protocols in the first group can't be used in real life, so they're useful for learning about noisy channel protocols. The classifications as seen in Figure 25.

There is a distinction between the protocols we address here and the protocols used in actual networks. The data frames pass from one node, referred to as the sender, to another node, referred to as the receiver, in all of the protocols we address.

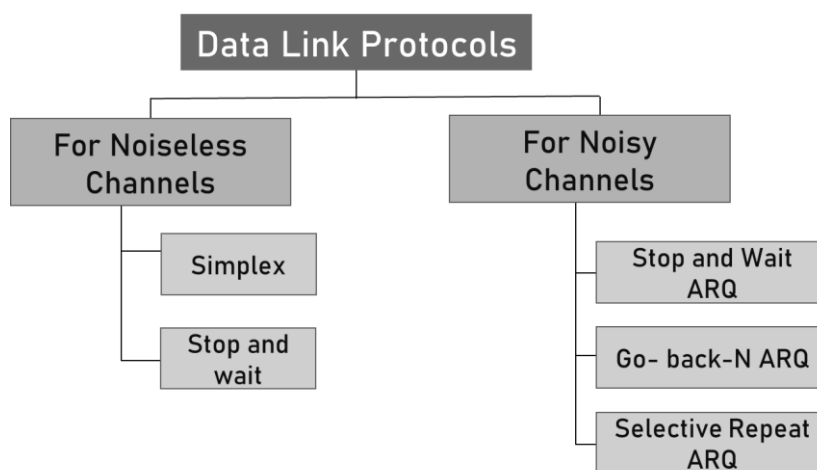


Figure 25 Classifications of protocols

Data flows in just one direction, despite the fact that special frames called acknowledgment (ACK) and negative acknowledgment (NAK) will flow in the opposite direction for flow and error monitoring. In a real-world network, data link protocols are bidirectional, allowing data to flow in all directions. Piggybacking is a method used in these protocols to provide flow and error control information such as ACKs and NAKs in the data frames. We picked the latter for our topic because bidirectional protocols are more complicated than unidirectional protocols. They can be generalised to bidirectional protocols if they are understood.

6.7 NOISELESS CHANNELS

Let's pretend we have an ideal channel with no missing, duplicated, or compromised frames. For this sort of channel, we add two protocols. The first is a protocol that does not use flow control, while the second does. Of necessity, neither has error management since we assumed the channel to be fully noiseless.

Simplest Protocol

For lack of a better term, we call our first protocol the Simplest Protocol. It has no flow or error management. It is a one-way protocol, meaning data frames only pass in one direction, from sender to receiver. We assume that the receiver can accommodate any frame it receives right away, with a loading time that is negligible. The receiver's data link layer separates the header from the frame right away and hands the data packet to its network layer, which will accept it right away. To put it another way, the receiver should never be overloaded with pictures.

Design

Flow control is not needed in this scheme. The sender site's data link layer receives data from its network layer, creates a frame out of it, and transfers it. The receiver site's data link layer collects a frame from its physical layer, removes data from it, and sends it to its network layer. The sender and receiver's data link layers provide transmission facilities to their network layers. For the actual exchange of bits, the data link layers depend on the services offered by their physical layers (such as signalling, multiplexing, and so on). A design as seen in Figure 26.

We need to go into the protocol that all data link layers use. The sender location would not be able to transmit a frame until it receives a data packet from its network layer. Until a frame arrives, the recipient location cannot send a data packet to the network layer. If the protocol is applied as a process, the concept of events must be introduced. The procedure at the sender site is still running; no action is taken before the network layer makes a request. The process at the receiver site is still running, but no action is taken before the physical layer sends a warning. Since they don't know when the associated incidents will occur, both operations are still going.

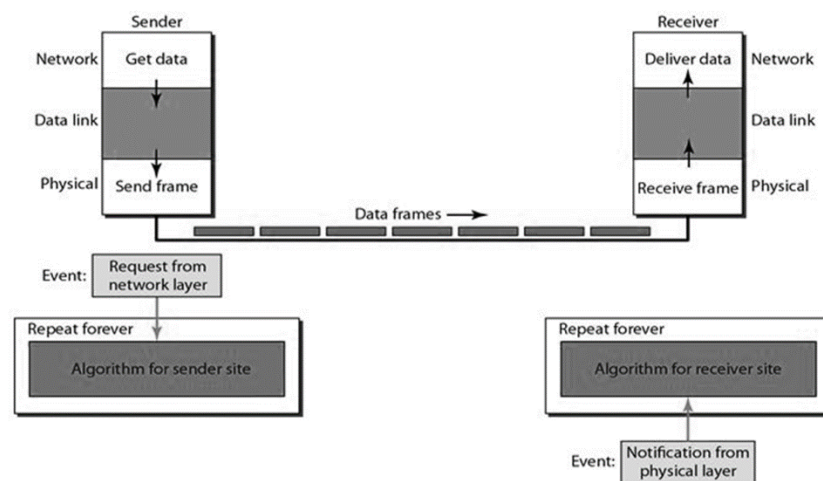


Figure 26 The design of the simplest protocol with no flow or error control

Algorithms

The protocol at the sender site is depicted in Algorithm 1.

Algorithm 1 The simplest protocol has a sender-site algorithm.

```

while(true)                                     //repeat forever
{
    WaitForEvent();                             //Sleep until an event occurs
    if(Event(RequestToSend))                    //There is a packet to send
    {
        GetData();                             //takes a data packet from
                                                the network layer
    }
}

```



```

        MakeFrame();                //adds a header and
                                    //delimiter flags to the data
                                    //packet to make a frame

        SendFrame();                //delivers the frame to the
                                    //physical layer for
                                    //transmission.
    }
}

```

Algorithm 2 The simplest protocol has a receiver-site algorithm.

```

while(true)                        //repeat forever
{
    WaitForEvent();                //Sleep until an event occurs
    if(Event(Arrivalnotification)  //There is a packet to send
    {
        ReceiveFrame();            //receives the frame from the
                                    //physical layer

        ExtractData();              //extracts the data from the
                                    //frame

        Deliverdata();              //delivers the data to the
                                    //network layer.
    }
}

```

Stop-and-Wait Protocol

When data frames arrive at the receiver site quicker than they can be processed, they must be stored before they are needed. The recipient, in most cases, does not have enough storage capacity, particularly if it is processing data from multiple sources. Frames may be discarded or service may be denied as a result of this. To keep the recipient from becoming overburdened for pictures, we need to advise the sender to slow down in some way. There has to be more. The sender receives input from the recipient. The Stop-and-Wait Protocol gets its name from the fact that the sender sends one frame, waits for clarification from the recipient (all right, go ahead), and then sends the next frame. Data frames remain unidirectional, but auxiliary ACK frames (simple tokens of acknowledgment) are sent in the opposite direction.

Example

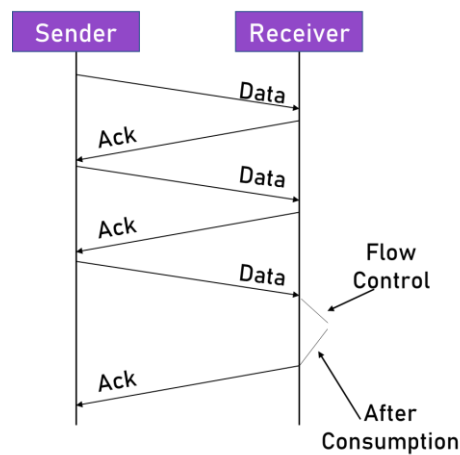


Figure 27 Stop and Wait Protocol

Figure 27 plays a collaboration sample using this protocol. It's all quite straightforward. The sender sends one frame and waits for the receiver's answer. When the ACK comes up, the next frame is sent by the sender. It's worth noting that sending two frames in the protocol puts the sender in danger. The sender is involved in four cases, while the recipient is involved in two.

6.8 NOISY CHANNELS

Since the Stop-and-Wait Protocol demonstrates how to incorporate flow control into it, Noiseless channels, unlike their predecessors, do not exist. We may choose to dismiss the mistake (as we always do). We either need to apply error management to our protocols (which we don't), or we need to add error control to our protocols.

Stop-and-Wait Automatic Repeat Request

The Stop-and-Wait Automatic Repeat Request (Stop-and-Wait ARQ) is our first protocol, which applies a basic error control feature to the Stop-and-Wait Protocol. Let's take a look at how this protocol does error detection and correction. We need to apply redundancy bits to our data frame to find and repair corrupted frames. Frames that have been lost are more difficult to deal with than those that have been corrupted. In previous protocols, we used There was no way to tell what frame it was. It's possible that the obtained frame is the right one. or a copy, or an out-of-order frame To solve the problem, number the frames. When it comes to the When a recipient gets an out-of-order data frame, it means the frames were either sent in the wrong order or were sent in the wrong order. duplicated or misplaced.

In this protocol, the completed and lost frames must be resent. If the recipient agrees, If the sender does not react when an error occurs, how can the sender know which frames to resend? To To avoid this, the sender saves a copy of the sent frame. It begins at the same time. Since an ACK frame can be corrupted and lost, it needs redundancy bits as well. as well as a series number This protocol's ACK frame includes a sequence number region. In this protocol, the sender actually discards or rejects a compromised ACK frame. One that is out of order.

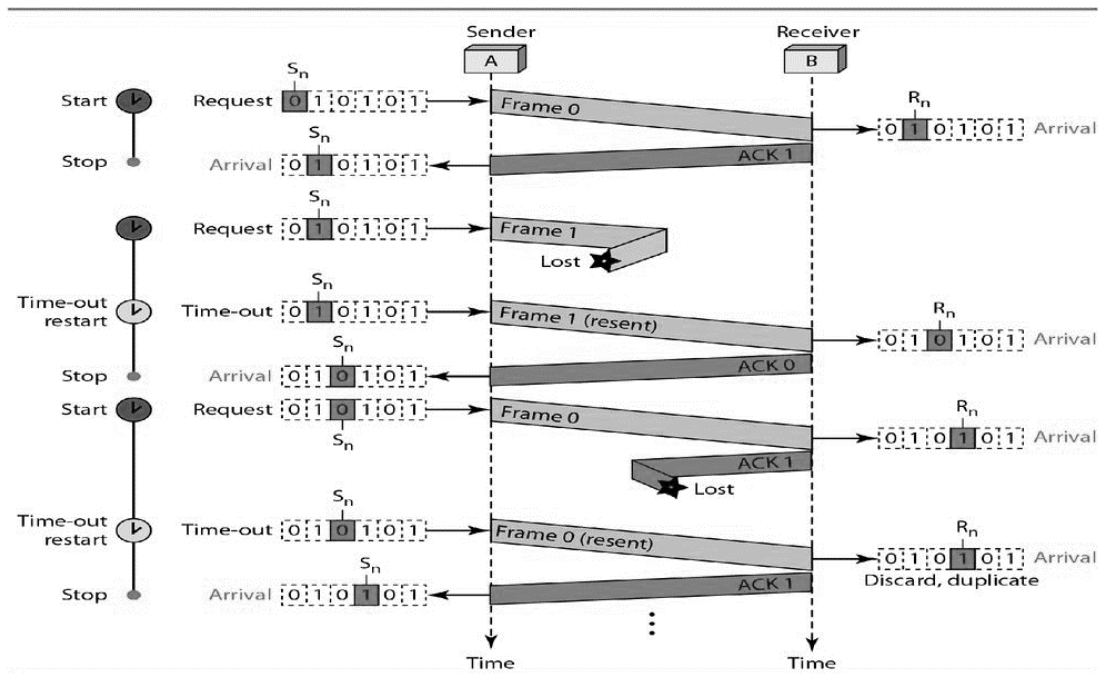


Figure 28 Stop and Wait ARQ

In Figure 28, frame an is submitted and accepted as an indication of Stop-and-Wait ARQ. After the time-out, frame 1 is missed and resent. The timer is stopped after the received frame 1 is acknowledged. The frame has been submitted and accepted, but the acknowledgement has been misplaced. Since the sender has no way of knowing if the frame or the acknowledgment has been lost, it resends frame 0, which is accepted, after the time-out.

Go-Back-N Automatic Repeat Request

Many frames must be in transition when waiting for acknowledgement to increase transmission performance (filling the pipe). To put it another way, we need to keep the channel busy while the sender waits for acknowledgement by allowing several frames to be outstanding. Go-Back-N Automatic Repeat Request is the only . We will send several frames in this protocol before collecting acknowledgements, and we hold a backup of these frames before the acknowledgements arrive.

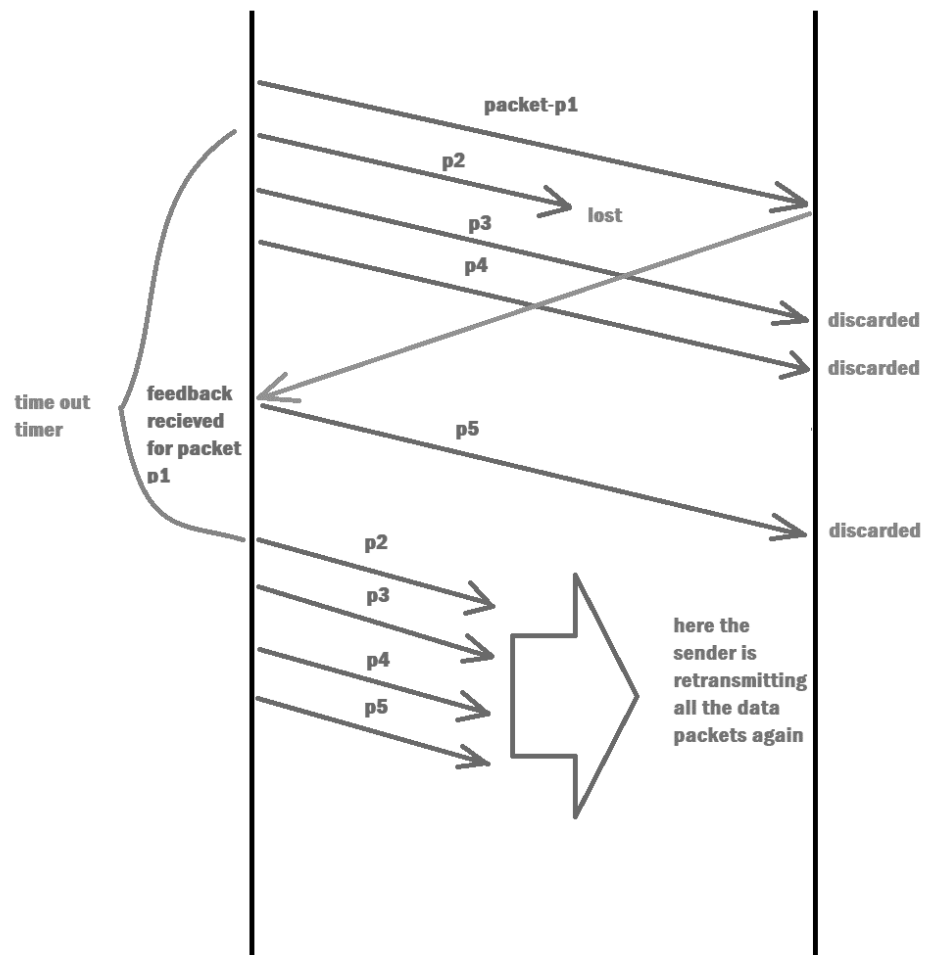


Figure 29 Go-Back-N ARQ

Selective-Repeat ARQ

The go-back-n protocol works well when errors are low, but it loses a lot of bandwidth on retransmitted frames when the line is poor. The selective repeat protocol is an alternative technique that allows the receiver to accept and buffer frames after a broken or missing one.

discerning Retransmit only those packets that are currently missing (due to errors) a second time:

The receiver must be able to accept out-of-order packets.

Since packets must be released in order to the higher layer, the recipient must be able to buffer any packets.

Retransmission requests:

Implicit – The receiver accepts any positive packet; packets that do not receive an ACK before the timeout are considered to be missing or incorrect. It's worth noting that this method must be used to ensure that any packet is obtained at some stage.

Explicit NAK (selective reject) – An explicit NAK (selective reject) can only request the retransmission of one packet. This method might speed up the retransmission, but it isn't strictly necessary. In reality, one or both methods are used.

Selective Repeat Protocol

Except that buffers are used and the recipient and sender each retain a window of space, this protocol (SRP) is almost identical to the GBN protocol. Where the connection is extremely unreliable, SRP works well.

Sender's Windows (Ws) = Receiver's Windows (Wr).

- In the SR protocol, the window size should be smaller than or equal to half the sequence number. This is to prevent packets from being misidentified. If the window size is greater than half the sequence number space, the sender can send new packets that the receiver interprets as retransmissions if an ACK is lost.
- The sender will send new packets as long as the total number of unacknowledged packets is less than W.
- After a timeout – or a NAK if NAK is used – the sender retransmits un-ACKed packets.
- The right packets are acknowledged by the receiver.
- The receiver saves the right packets before they can be sent to the higher layer in time.
- The sender and receiver windows in Selective Repeat ARQ must be no more than one-half of a metre wide.

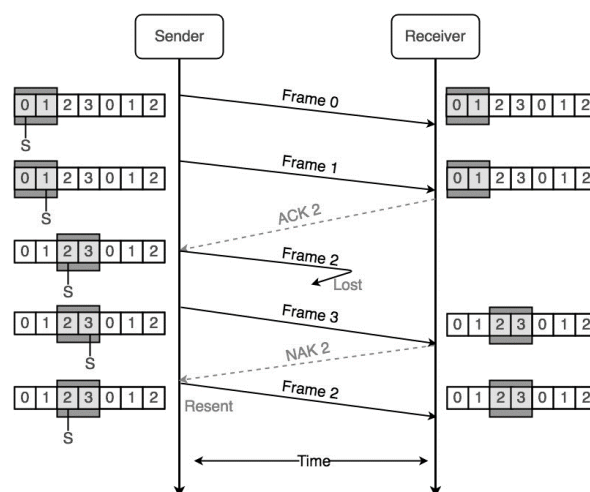


Figure 30 Selective Repeat Protocol

Figure 30: Only frames for which a NAK is sent are retransmitted by the sender.

The Selective Repeat Protocol (SRP) has the same utility as the GO-Back-N protocol.

$$\text{Efficiency} = N / (1 + 2a)$$

Where a = Propagation delay / Transmission delay

$$\text{Buffers} = N + N$$

$$\text{Sequence number} = N \text{ (sender side)} + N \text{ (Receiver Side)}$$

Piggybacking

Data frames flow in only one direction in the three protocols we discussed in this section, though control information such as ACK and NAK frames will pass in both directions. Data frames flow in both ways in real life: from node A to node B and from node B to node A. This necessitates the flow of control knowledge in both directions. Piggybacking is a method for increasing the reliability of bidirectional protocols. When a frame carries data from point A to point B, it can also carry control information about arrived (or lost) frames from point B; when a frame carries data from point B to point A, it can also carry control information about arrived (or lost) frames from point A.

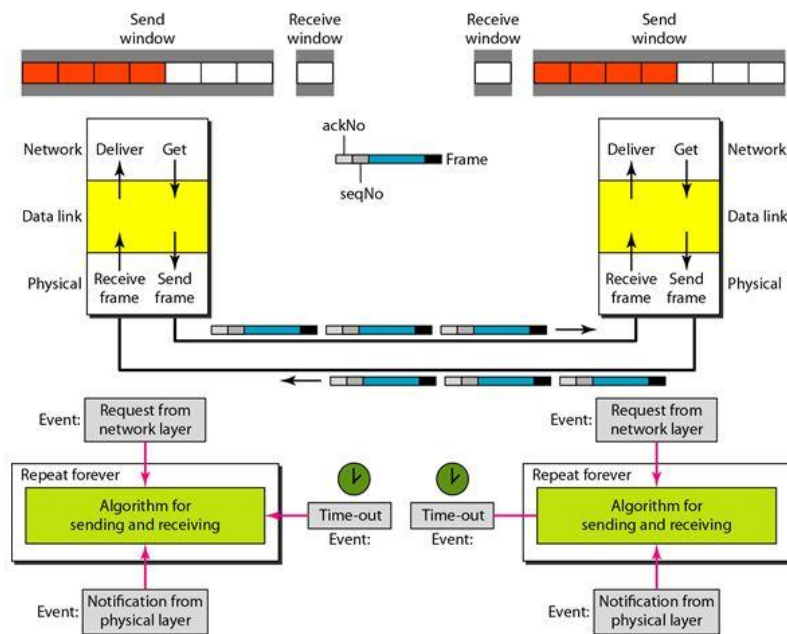


Figure 31 Design of piggybacking in Go-Back-NARQ

Figure 31 shows a specification for a Go-Back-N ARQ that uses piggybacking. Each node now has two windows: one for sending and one for receiving data. Both include the use of a timer. Request, entry, and time-out are three forms of incidents in which all are concerned. The arrival case, on the other hand, is complicated; when a frame arrives, the site must manage all control details and the frame itself.

All of these issues must be addressed in a single situation: the arrival event. At each location, the request event only uses the send window; the arrival event requires both windows. The fact that all pages would use the same algorithm is a vital aspect of piggybacking. Since it would merge two arrival events into one, this algorithm is difficult.

Self-Assessment

- Which can be used as an intermediate device in between transmitter entity and receiver entity?
 - IP router
 - Microwave router
 - Telephone switch
 - All of the mentioned
- Which is more efficient?
 - Parity check
 - Cyclic redundancy check
 - Parity & Cyclic redundancy check
 - None of the mentioned
- CRC uses
 - Multiplication
 - Binary division
 - Multiplication & Binary division
 - None of the mentioned
- In _____ coding, we divide our message into blocks, each of k bits, called _____.

- a) block; blockwords
 - b) linear; datawords
 - c) block; datawords
 - d) none of the above
5. Checksums use _____ arithmetic.
- a) one's complement arithmetic
 - b) two's complement arithmetic
 - c) either (a) or (b)
 - d) none of the above
6. The checksum of 1111 and 1111 is _____.
- a) 0000
 - b) 1111
 - c) 1110
 - d) 0111
7. In cyclic redundancy checking, the divisor is _____ the CRC.
- a) one bit less than
 - b) one bit more than
 - c) The same size as
 - d) none of the above
8. In cyclic redundancy checking, what is the CRC?
- a) The quotient
 - b) The dividend
 - c) The divisor
 - d) The remainder
9. Stop and Wait Automatic Repeat request is a special case of
- a) simplest protocol
 - b) Go Back-N Automatic Repeat request
 - c) Selective Repeat Automatic Repeat Request
 - d) stop and wait
10. The send window in the Go-Back-N Protocol is an abstract conceot defining an imaginary box with
- a) one variable
 - b) two variables
 - c) three variables
 - d) four variables
11. The stop and wait uses the link of
- a) modulation
 - b) full duplex
 - c) half duplex
 - d) de modulation
12. In block coding, we divide our message into blocks, is called
- a) code blocks

- b) packet blocks
- c) code words
- d) data words

Answers: Self-Assessment

- | | | | | | | | |
|----|---|-----|---|-----|---|-----|---|
| 1. | d | 2. | b | 3. | c | 4. | c |
| 5. | a | 6. | a | 7. | b | 8. | d |
| 9. | b | 10. | c | 11. | c | 12. | d |

Summary

- The strategies for gaining access to a shared communication channel and ensuring secure data transfer are described in the data link layer. Framing, checksums, error detection and correction, acknowledgment, flow control, and encapsulating packets from the network layer to frames are some of its key responsibilities.
- Unacknowledged connectionless service, acknowledged connectionless service, and acknowledged connection-oriented service are all provided by the data link layer.
- The simplest type of error detection is parity check, which requires the receiver to count only the number of 1s in the obtained data stream with an additional parity bit.
- A checksum is a basic redundancy scan that is used to find data errors.
- Cyclic Redundancy Check is a method for adding a data string to packets of information that can be used to detect errors in the data packets that is commonly used in computer networks.
- On an error-free communication channel, the Stop and Wait protocol is the simplest to adopt and proves to be the most effective. An error-free contact channel, on the other hand, is virtually impossible.
- The Go Back N protocol necessitates buffer management, making it difficult to maintain source and destination devices in line. It's actually the least effective since it retransmits all subsequent frames in the event of a frame failure, wasting a lot of bandwidth.
- Selective Repeat is an enhancement of the Go Back N protocol that attempts to make more usage of bandwidth by reducing the amount of retransmissions by retransmitting just one frame rather than the whole sequence. As a result, Selective Repeat is a safer option.

Keywords

The Point-to-Point Protocol (PPP) is a data link layer protocol that links two communicating link-level peers at either end of a point-to-point link.

Selective Repeat: Provides buffers at the source and destination hosts, allowing the source node to provide multiple remaining frames at the same time and the destination node to accept out-of-order frames and store them in its window.

Simplex is a term that refers to Stop and Wait: After propagation, the source node waits for the destination node to accept it. Following receipt of the acknowledgment, the loop is repeated again

Checksum: An algorithm for calculating the binary values in a packet or other block of data and storing the results with the data to correlate with a new checksum at the other end.

Cyclic Redundancy Check: A method for adding a data string to information packets that can be used to find errors in the data packets.

Error management entails sequencing frames and submitting confirmation control frames.

Controlling the rate of data transfer between two source and destination hosts is referred to as flow control.

Framing: The data link layer splits the bit stream into frames to provide a secure transmission of bit streams to the network layer.

Go Back N: Using buffers, the Go Back N protocol allows the source computer to have multiple outstanding frames at the same time.

Even parity and odd parity methods are used in parity checks. The receiver's procedure is straightforward since it just has to count the number of 1s in the obtained data stream with the inclusion of a parity bit.

Acknowledged Connectivity-oriented Operation: Before any data transfer, the data link layer offers this service to the network layer by creating a connection between the source and destination hosts.

Acknowledged Connectionless Service: When each frame transmitted between two hosts is sent correctly, it is referred to as acknowledged connectionless service.

Review Questions

1. What is the data link protocol?
2. What advantages does Selective Repeat sliding window protocol offer over Go Back N protocol?
3. What is the purpose of flow control?
4. Describe how does finite state machine model carry out protocol verification?
5. What are different data link protocols available? Why does PPP have become popular?
6. Explain error detection techniques.
7. Explain hamming code with example.

Further/Suggested Readings



- Andrew S. Tanenbaum, *Computer Networks*, Prentice Hall.
- Behrouz A. Forouzan and Sophia Chung Fegan, *Data Communications and Networking*, McGraw-Hill Companies.
- Burton, Bill, *Remote Access for Cisco Networks*, McGraw-Hill Osborne Media, McGraw-Hill Osborne Media.
- Rajneesh Agrawal and Bharat Bhushan Tiwari, *Computer Networks and Internet*, Vikas Publication
- <https://www.geeksforgeeks.org/computer-network-tutorials/>

